CORE ATTENDEE EDITION: Start with 00_START_HERE.md at the pack root. Only the core two-day route is supplied. Optional publishing, dummy website, resource creation, identity/networking and advanced Azure tracker extensions mentioned below are not assigned or included. Use your own extracted path where a command shows C:\YOUR-ATTENDEE-PACK. Do not copy instructor settings.

KQL - first rows to a correlated incident

LBS | Attendee edition | Fictional data only | Updated 28 October 2026

Before you start

Expected duration: 75 minutes: 13:30-14:45

Prerequisites: Application Insights walkthrough; supplied Labs\Day2Operations\Queries and CSVs.

Required permissions: Approved log query rights for live or prepared-query mode. Offline CSV review needs no Azure access.

Resources created: None. Prepared datatables do not ingest rows or create persistent tables.

Cost considerations: Filter live queries to the approved app and recent time window. Query charges depend on table plan/features; do not assume all monitoring is free.

Safety: Use the assigned tenant, subscription and resource only. Do not change shared networking, roles, tiers or deployment settings. No real documents, secrets or keyed URLs in evidence.

Select a route before running a query

Workspace schema: Queries\Workspace uses AppRequests/AppDependencies, TimeGenerated, DurationMs and OperationId. Replace YOUR-APPLICATION-INSIGHTS-RESOURCE-ID with the component's full resource ID, not its workspace GUID.

Application Insights component schema: Queries\ApplicationInsights uses requests/dependencies, timestamp, duration and operation_Id. The supplied files normalise duration to milliseconds and names for comparison. Confirm the current component; do not run across unrelated apps.

Prepared query: Queries\Prepared defines fictional Requests/Dependencies inside each file. Run each complete file separately in an authorised KQL editor. Fixed data timestamps are 28 October 2026, 10:00-10:04 UTC; set a custom portal time range that includes them. No ago() filter is used on the prepared data.

Offline: open the two fictional telemetry CSVs and predict the rows. Label it offline review; it is not KQL execution or a live monitoring pass.

Meaning	Workspace	Application Insights
Requests	AppRequests	requests
Dependencies	AppDependencies	dependencies
Time	TimeGenerated	timestamp
Duration	DurationMs (number, ms)	duration (timespan; divide by 1ms)
Correlation	OperationId	operation_Id
Success / result	Success / ResultCode	success / resultCode

Three levels of practice

1. Level 1: open 01-first-requests.kql from the chosen route. In Logs, close the hub, switch Agent off, choose KQL mode and run the whole file. For a live workspace, the starting idea is AppRequests | take 10 with time/app filters added.

Expected result: Prepared mode: five rows ordered by time. Live mode: up to ten rows; actual counts vary.

2. Read column names, data types, status code and duration. Save a non-secret copy of the query text locally if central saving is not permitted.

Expected result: You can explain where the request ran and what this result does not prove.

3. Level 2: run 02-failed-requests.kql. Inspect Success == false; do not confuse a business status such as Needs review with an HTTP failure.

Expected result: Prepared mode: POST /documents 500 op-003 and GET /private-status 401 op-005. Live mode: inspect actual returned failures, possibly none.

4. Level 3: run 03-slow-dependency-correlation.kql. Explain the teaching threshold DurationMs > 1000. The leftouter join adds request details with matching OperationId; missing matches remain visible.

Expected result: Prepared mode: two rows, op-003 then op-004, dependency durations 5100 and 4600 ms. A slow success and a slow failure require different reasoning; both merit investigation.

5. Choose one operation, construct a UTC timeline and write observation, hypothesis, next read-only check and recovery verification. Do not change configuration in this exercise.

Expected result: A reproducible incident note. Correlation alone does not prove why the dependency was slow.

How to read the query

A pipe passes a table to the next operator. where filters rows; project chooses/renames columns; order by sorts; take limits output. The let statements define named query expressions. Run the entire file so those definitions are in scope.

A join can produce more than one row per operation if several related events share an ID. Our prepared dataset has one dependency and one request per demonstrated operation, not a general production guarantee.

count() measures stored rows, not necessarily total traffic under sampling. Inspect ItemCount and instrumentation before estimating traffic. Our prepared dataset contains no sampling and is not production evidence.

Troubleshooting - symptom, checks and safe recovery

Symptom	Exact checks / recovery
Unknown table/column	Confirm schema and use the matching directory; do not randomly change case or mix duration and DurationMs.
Unknown Requests name	Select and run the complete file including let definitions, not only its final lines.
Prepared result empty	Include 28 October 2026 10:00-10:05 UTC in the portal time picker. Use the prepared file without live ago() filters.
No correlated request	Check actual OperationId, time window, resource scope and sampling. Keep the unmatched dependency; do not fabricate a row.
Slow query / huge results	Confirm bounded time and app filters. Avoid unbounded union/search across all tables; take limits output, not necessarily scanned data.

Knowledge check - before and after

Before: does an empty failure table prove an app is healthy? What is the difference between a table name and a workspace ID?

After: find the two prepared failures, explain the duration units and identify the slow successful operation. Why does correlation suggest a next check rather than prove root cause?

Cleanup

Keep useful query text and approved redacted notes. No table deletion is required: prepared let/datatable values are query-local.

Do not create a query pack/workbook or change retention without separate approval. Record central saving as Not run if not attempted.

Your evidence

Record	Your result
Mode: live / prepared query / offline review	
Scope, time range and source	
Expected / actual result	
Passed / Blocked / Not run and why	
Cost/security trade-off	
Follow-up owner / cleanup	

Official schema references

https://learn.microsoft.com/azure/azure-monitor/reference/tables/apprequests

https://learn.microsoft.com/azure/azure-monitor/reference/tables/appdependencies